IA POUR LES SYSTÈMES EMBARQUÉS

Une lampe qui comprend. Même sans Internet.

Comprendre l'IA embarquée, du premier besoin aux défis de recherche.

Roland Cédric TAYO

3 heures · Six chapitres · Questions et expériences
Alice est fictive. Les découvertes sont documentées.



Avant de commencer, qu'en pensez-vous ?

Quatre questions pour partir de vos idées. Nous y reviendrons à la fin de l'histoire.

01

Quel appareil de votre quotidien utilise, selon vous, de l'IA ?

02

Une IA peut-elle fonctionner sans Internet ?

03

Pourquoi une IA qui marche sur PC pourrait-elle poser problème sur une petite carte ?

04

Comment vérifier qu'une modification apporte un progrès réel ?

Proposez une réponse avec vos mots. Gardez-la en tête pour la fin du cours.

Alice voudrait parler à sa lampe.

Elle peut déjà appuyer sur un bouton. Pourquoi ajouter une commande vocale ?

> *« Lumière, s'il te plaît ! »*
>
> Alice

- Quand ses mains sont occupées, la voix pourrait lui rendre service.
- La lampe devrait reconnaître une commande, même dite autrement.
- Si un bouton suffit au besoin, il reste une bonne solution.

L'IDÉE À RETENIR

On commence par ce que la personne veut faire, puis on choisit la technique.



Un bouton fonctionne. La voix pourrait rendre un autre service.

Vous dites le même mot. L’appareil reçoit des sons différents.

Alice essaie sa voix, celle d’un ami, puis la même phrase avec du bruit.

- Une voix plus grave, un mot plus rapide : les nombres changent.
- Comparer avec un unique enregistrement ne suffit pas.
- On peut apprendre à reconnaître des régularités dans des exemples.

L’IDÉE À RETENIR

Apprendre aide à traiter des variations ; réussir sur des sons nouveaux reste à vérifier.



Le modèle reçoit des nombres, pas des mots écrits.

Trois personnes. Un même mot. Des sons différents.

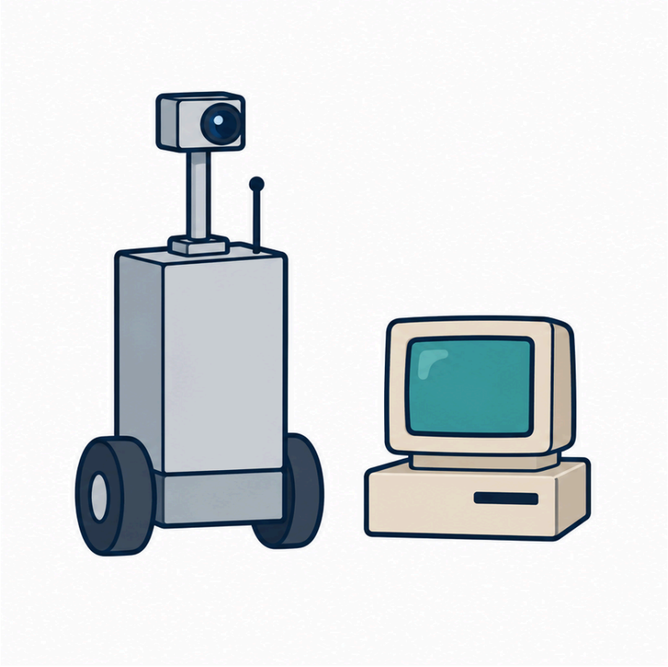
1966 : faire sortir l'IA de la feuille de papier.

À SRI, Charles Rosen et l'équipe de Shakey veulent faire agir un robot dans une pièce.

- Le robot observe, prépare une action, puis se déplace.
- Il échange avec un ordinateur extérieur qui réalise le calcul.
- La question est déjà là : que doit faire la machine, et où calculer ?

L'IDÉE À RETENIR

Une machine qui agit dans le monde ne fait pas forcément tous ses calculs à bord.



Le robot observe et agit. L'ordinateur extérieur participe au calcul.

Reconstitution pédagogique de Shakey · SRI, 1966–1972. Ce n'est pas une photographie d'archive.

1989 : Dean Pomerleau apprend à suivre une route.

Avec ALVINN, il étudie comment passer d’observations à une direction.

- Le réseau reçoit des observations et produit une direction.
- Ses poids sont ajustés sur des exemples, au lieu d’écrire chaque situation.
- Les essais portent sur des conditions précises. Ils ne prouvent pas une conduite autonome générale.

L’IDÉE À RETENIR

Un réseau apprend une relation entre ce qu’il reçoit et ce qu’il doit produire.



Observer la route → calculer avec le réseau → proposer une direction.

Illustration du principe d’ALVINN · Dean Pomerleau, 1989. Véhicule reconstitué, pas une archive.

Comment vérifier que la lampe reconnaît d’autres voix ?

La lampe a appris le mot « lumière » à partir de plusieurs enregistrements. Quel test faut-il choisir ?



Choisissez le test qui met le modèle face à de nouvelles personnes.

POUR POURSUIVRE APRÈS LA RÉPONSE

- Quels bruits faudrait-il aussi essayer ?

A Les voix qui ont servi à l’apprentissage.

B Des voix gardées à part pendant l’apprentissage.

C Un enregistrement déjà appris, diffusé plus fort.

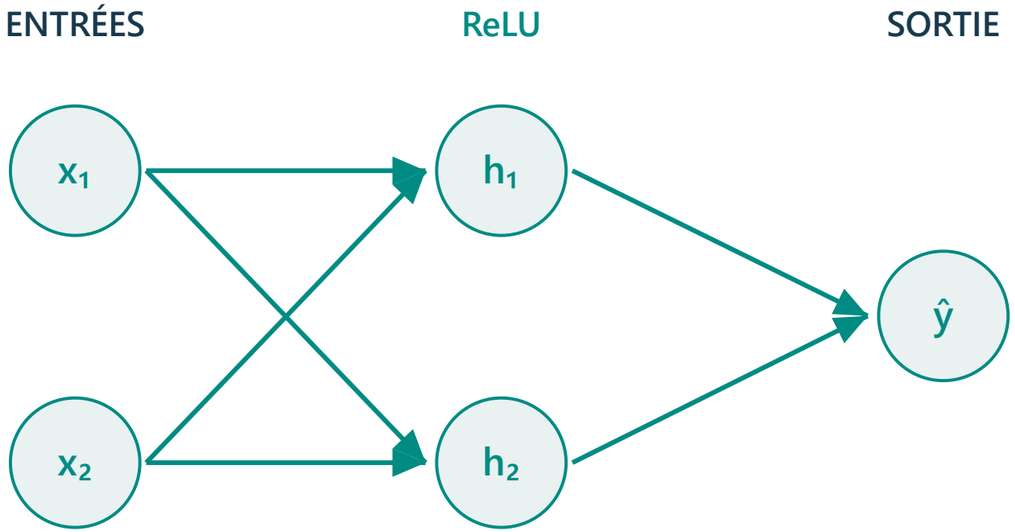
Que fait le modèle avec des nombres ?

Prenons un tout petit réseau pour suivre chaque calcul. Ce réseau ne reconnaît pas la voix.

- Un **poids** multiplie une entrée. Un **biais** ajoute un décalage.
- **ReLU** conserve un nombre positif et remplace un nombre négatif par zéro.
- Faire une **inférence**, c'est calculer la sortie avec des poids déjà fixés.

L'IDÉE À RETENIR

Un réseau est une suite de calculs. On peut les suivre un par un.



6 poids + 3 biais = 9 paramètres

2 × 2 + 2 × 1 = 6 MAC, biais exclus

Calculons une sortie, ensemble.

x_1 et x_2 sont les deux entrées. h_1 et h_2 sont les résultats des deux neurones cachés.

- $h_1 = \max(0, x_1 - x_2)$. Avec 1 et 2, on obtient 0.
- $h_2 = \max(0, 0{,}5x_1 + 0{,}5x_2)$. On obtient 1,5.
- $\hat{y} = 2h_1 + h_2 = 1{,}5$. Le chapeau de ŷ signifie « valeur prédite ».

L'IDÉE À RETENIR

Changer les entrées change la prédiction ; les poids restent identiques.

INFÉRENCE · POIDS FIXES

x_1 **1**

x_2 **2**

h_1	h_2
0	**1,5**

$\hat{y} =$ **1,5**

h_1, h_2 : activations après ReLU. Les biais sont nuls ici ; aucun poids ne change.

La réponse est 1,5. On attendait 2.

Comment corriger le réseau ? Essayons sur un seul poids de sortie, nommé w.

- $L = (\hat{y} - c)^2$: la **perte** L mesure l'écart avec la cible c, ici 2.
- $g = 2(\hat{y} - c)h_2 = -1{,}5$: ce **gradient** indique comment la perte varie avec w.
- $w \leftarrow w - \eta g$. Avec un pas $\eta = 0{,}1$, w passe de 1 à 1,15.

L'IDÉE À RETENIR

Apprendre consiste à corriger les poids pour réduire une erreur mesurée.

UNE CORRECTION · UN SEUL POIDS

Pas η **0,1**

Poids après correction	Nouvelle sortie ŷ
1,15	**1,725**

Perte avant	Perte après
0,25	**0,075625**

La perte diminue sur cet exemple.

$h_2 = 1{,}5$ et cible $c = 2$ sont fixes. Une correction n'est pas un entraînement complet.

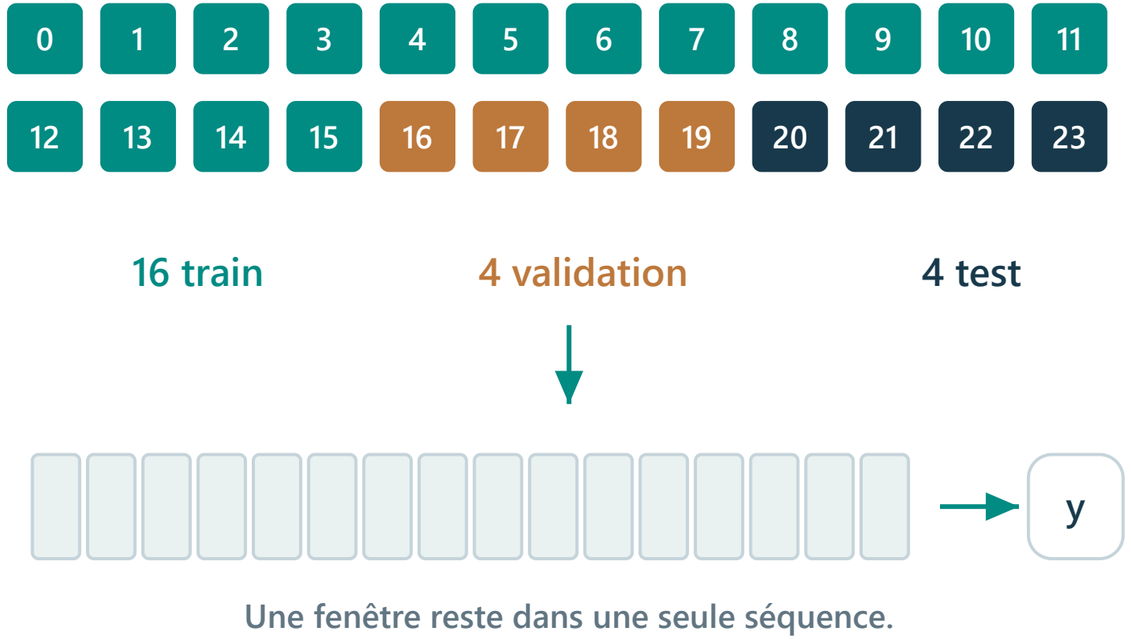
Elle connaît les exemples. A-t-elle compris la tâche ?

Alice ne peut pas juger le modèle uniquement avec les données qui ont servi à le régler.

- L'**apprentissage** sert à ajuster les poids et la normalisation.
- La **validation** sert à choisir les réglages. Le **test** reste à part jusqu'à l'évaluation finale.
- Pour un signal, séparer les séquences avant de découper des fenêtres évite de partager les mêmes mesures.

L'IDÉE À RETENIR

Le test doit poser des questions nouvelles au modèle.



Une fenêtre reste dans une seule séquence.

Quelle est la sortie du réseau ?

Le même petit réseau reçoit $x_1 = 2$ et $x_2 = 1$. Ses poids n'ont pas changé.

Les calculs du réseau

$h_1 = \max(0, x_1 - x_2)$

$h_2 = \max(0, 0{,}5x_1 + 0{,}5x_2)$

$\hat{y} = 2h_1 + h_2$

Calculez h_1 et h_2, puis choisissez la valeur de $\hat{y}$.

POUR POURSUIVRE APRÈS LA RÉPONSE

- Quels nombres changent pendant l'inférence : entrées, activations ou poids ?

A 1,5

B 2,5

C 3,5

Du son à la lumière, il faut toute une chaîne.

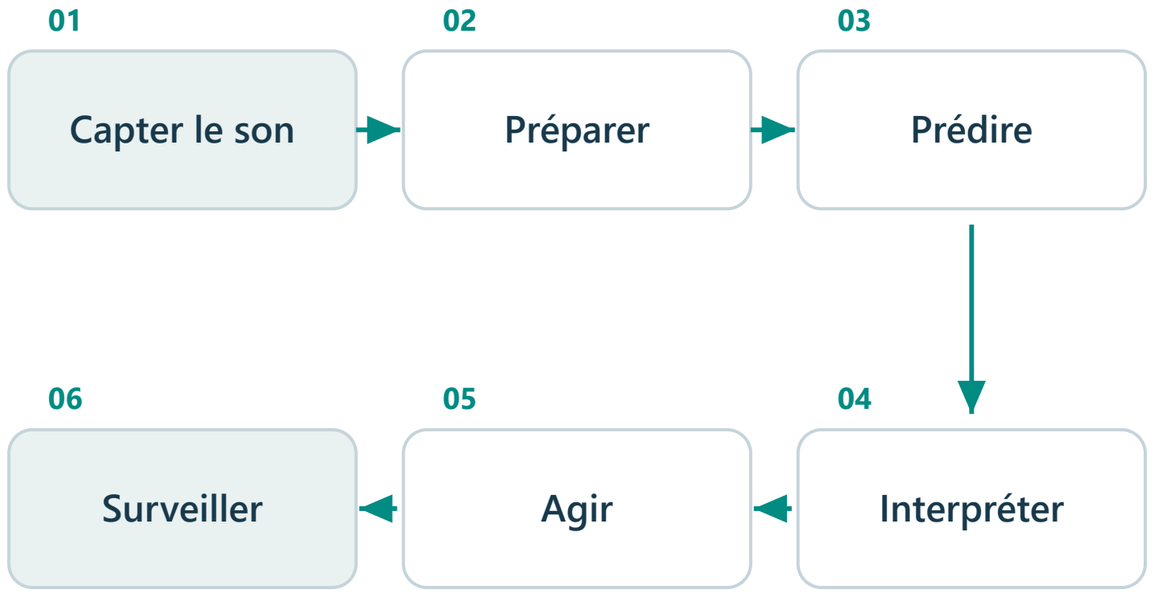
Pour son premier essai, Alice fait reconnaître la commande par un serveur.

> *« Avec le Wi-Fi, ça marche. »*
>
> Alice

- Le **capteur** fournit le son ; le **prétraitement** prépare les nombres attendus.
- Le **modèle** produit une prédiction ; une règle décide si la lampe doit agir.
- Il faut aussi une liaison disponible et un repli si la réponse manque.

L'IDÉE À RETENIR

Le réseau est une fonction dans un système complet.



Le modèle ne réalise pas toute l'application.

Le Wi-Fi tombe.
La lampe ne répond plus.

Alice répète le même mot. Le modèle n'a pas changé ; la liaison manque.

> *« Le bouton marche encore. Mais pas ma commande vocale. »*
>
> Alice

- La fonction attendue doit-elle marcher pendant la coupure ?
- Si oui, on peut exécuter la reconnaissance dans l'appareil.
- Une commande manuelle reste utile si la reconnaissance échoue.

L'IDÉE À RETENIR

Embarquer le calcul peut enlever une dépendance au réseau.





Pas de liaison → pas de réponse distante.

Le bouton change le schéma. Il ne contacte aucun serveur.

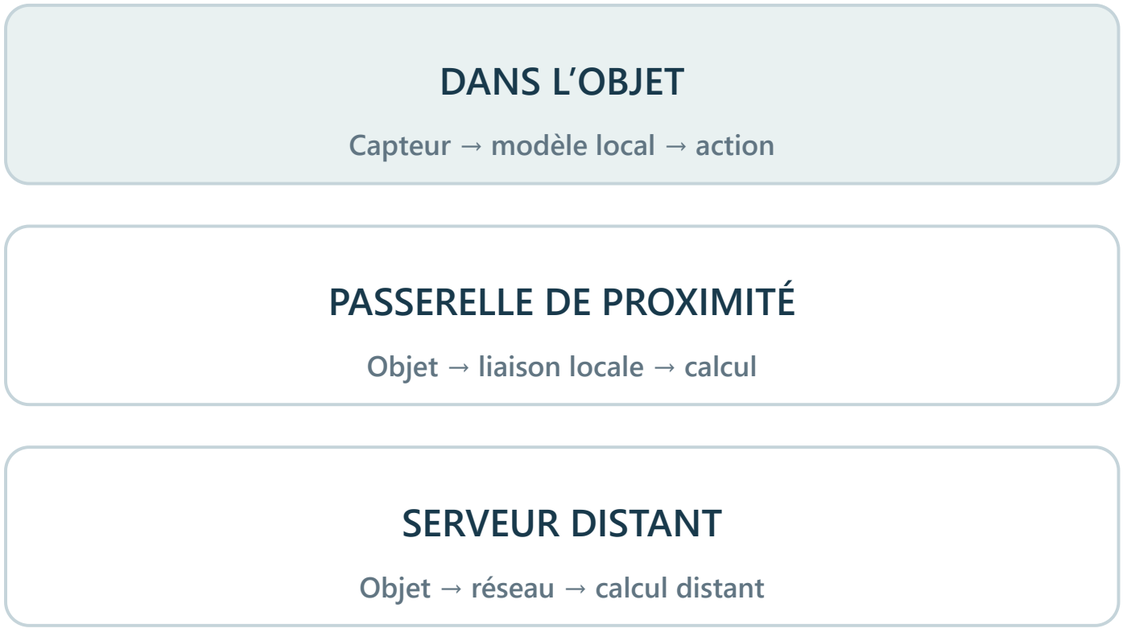
Où mettre le calcul ?

Dans l’objet, sur une passerelle voisine, ou sur un serveur : cela ne rend pas le même service.

- **Dans l’objet :** pas de liaison nécessaire pour cette inférence.
- **À proximité :** une passerelle calcule, avec une liaison locale.
- **À distance :** le serveur calcule, avec une liaison réseau.

L’IDÉE À RETENIR

On peut apprendre sur un PC et utiliser le modèle dans l’objet.



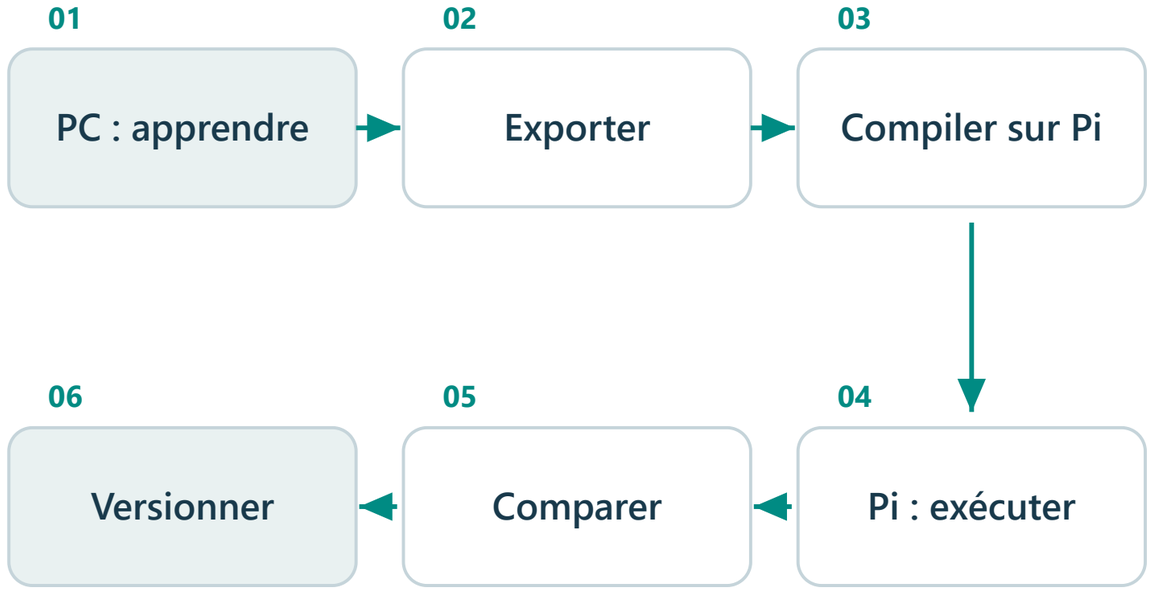
Qu’emporte-t-on du PC vers la carte ?

Les poids seuls ne suffisent pas : l’appareil doit refaire le même traitement.

- Exporter les **poids**, la **normalisation**, l’ordre des entrées et la configuration.
- Dans le TD : entraîner en Python sur PC ; compiler l’inférence C sur Raspberry Pi.
- Tester les mêmes entrées sur PC et Pi, puis conserver une version identifiée.

L’IDÉE À RETENIR

Déployer, c’est faire fonctionner le traitement complet sur la vraie cible.



Poids + préparation + programme + configuration

Où faut-il exécuter la reconnaissance ?

La commande doit fonctionner sans Internet. Aucun enregistrement audio ne peut quitter la lampe.

Deux exigences

1. Fonctionner sans Internet.
2. Garder l'audio dans la lampe.

Vérifiez les deux exigences, pas seulement la connexion à Internet.

POUR POURSUIVRE APRÈS LA RÉPONSE

- les mises à jour du modèle peuvent-elles encore venir d'un serveur ?

A Dans la lampe.

B Sur une passerelle Wi-Fi de la maison.

C Sur un serveur distant.

On reprend dans dix minutes.

La reconnaissance locale est une piste. Reste à la faire entrer dans la petite boîte.

- Nous savons ce que la lampe doit faire.
- Nous savons où elle doit le faire.
- Après la pause : avec quelle mémoire, quel délai et quelle batterie ?



Après la pause : comment le faire tenir et fonctionner ?

Une carte n’est pas un PC en miniature.

Le programme doit respecter ce que le matériel peut réellement exécuter et stocker.

- Un **microcontrôleur** réunit calcul, mémoire et périphériques pour une application.
- Un **Raspberry Pi** possède d’autres ressources et peut exécuter Linux.
- Un **GPU** ou un **NPU** accélère certains calculs, s’il existe sur la cible et les prend en charge.

L’IDÉE À RETENIR

Le même modèle peut convenir à une carte et être impossible à utiliser sur une autre.

MICROCONTRÔLEUR

CPU

Flash + SRAM

Périphériques

Application dédiée

SYSTÈME LINUX

CPU

RAM + stockage

OS + périphériques

Exemple : Raspberry Pi

NPU / GPU : accélérateurs éventuels du système

Vérifier le matériel réellement présent sur votre carte.

Le modèle est enregistré. Pourquoi ne démarre-t-il pas ?

Alice avait compté les poids. Elle avait oublié la place nécessaire pendant le calcul.

- Le **stockage** garde le programme et les poids, selon leur placement.
- La **RAM** accueille les entrées, les résultats intermédiaires et le travail du programme.
- Le **pic mémoire** est la plus grande quantité occupée au même moment.

L'IDÉE À RETENIR

La place du fichier et la place utilisée pendant le calcul sont deux budgets différents.

EXEMPLE MCU · POIDS CONSERVÉS EN FLASH

Poids bruts	Travail SRAM
400 Ko	**120 Ko**
100 000 paramètres	activations + buffers + pile



Poids FP32 | Poids INT8 seuls

Budgets fictifs distincts. Échelles et métadonnées exclues du payload ; SRAM inchangée dans cet exemple.

2017 : Howard et ses collègues réduisent le travail à faire.

MobileNets s’attaque au coût des convolutions pour la vision sur appareils mobiles.

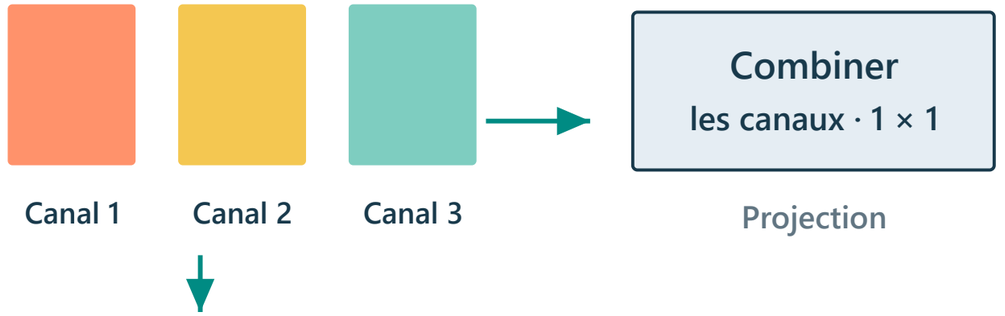
- Une **convolution** recherche des motifs locaux dans une image.
- Leur idée : filtrer chaque canal séparément, puis mélanger les canaux avec un calcul 1 × 1.
- Cela réduit les opérations. Le temps gagné dépend encore de la carte.

L’IDÉE À RETENIR

On peut changer la façon de calculer, plutôt que seulement utiliser un processeur plus puissant.

2017 · MobileNets

Andrew Howard et ses collègues



Moins d’opérations.

Le gain de temps reste à mesurer.

2018 : Jacob et ses collègues changent la façon d’écrire les nombres.

La quantification permet d’exécuter des réseaux avec des nombres entiers adaptés au matériel.

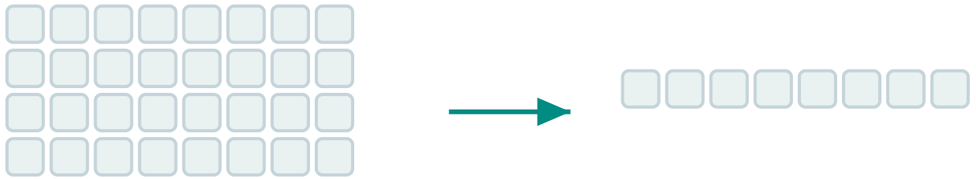
- FP32 utilise 32 bits par valeur ; INT8 utilise 8 bits, soit un octet.
- On représente une plage de valeurs avec une échelle et des arrondis.
- Il faut vérifier l’erreur, la qualité du modèle et les calculs réellement exécutés.

L’IDÉE À RETENIR

Des nombres plus courts prennent moins de place, mais ne décrivent plus exactement les mêmes valeurs.

2018 · Calcul entier

Benoit Jacob et ses collègues



32 bits par poids

8 bits par poids

4 fois moins de place pour ces poids.

Il faut aussi les échelles et des accumulateurs adaptés.

Que perd-on en arrondissant ?

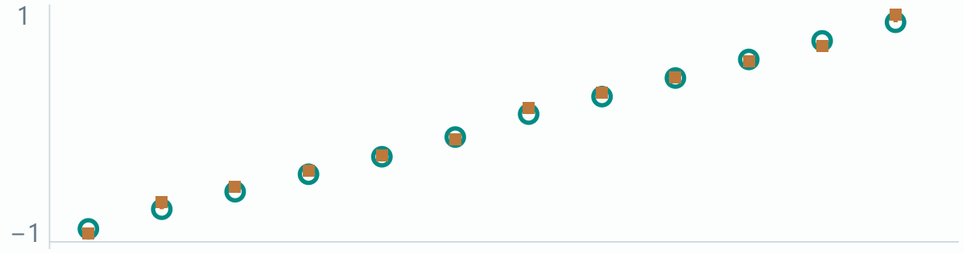
x est la valeur de départ. On la remplace par l'un des nombres que le format peut représenter.

- s est l'**échelle** : l'écart entre deux niveaux.
- q = arrondi(x/s) est le code entier, limité à la plage autorisée.
- $\hat{x} = s \times q$ reconstruit une valeur approchée. Comparer $\hat{x}$ et x mesure l'erreur.

L'IDÉE À RETENIR

Réduire le nombre de niveaux crée des arrondis ; il faut mesurer leur effet sur la tâche.

ARRONDI SYMÉTRIQUE · EXEMPLE SUR [−1, 1]



12 valeurs illustratives

Précision des codes **4 bits**

Pas s : **0,1429**

Erreur max illustrée : **0,07**

Cercles : valeurs initiales ; carrés : valeurs reconstruites. Pas un test de qualité de réseau.

Combien de RAM faut-il pendant ce calcul ?

Les poids restent en Flash. Le tableau décrit ce qui occupe la RAM au même moment.

Présents en RAM au même moment

Usage	Kio
Activations	60
Temporaires	40
Pile	20
Entrées-sorties	16

La carte dispose de 128 Kio de RAM.

Additionnez les quatre usages RAM du tableau.

POUR POURSUIVRE APRÈS LA RÉPONSE

- compresser seulement les poids Flash résout-il ce dépassement ?

A 120 Kio

B 136 Kio

C 216 Kio

Moins de calculs. Toujours de l’attente.

Le processeur doit aussi aller chercher les nombres et ranger les résultats.

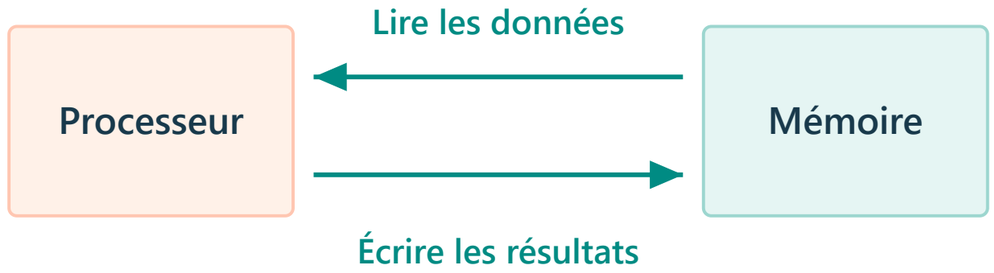
- Les **poids** et les **activations** doivent être lus, écrits et parfois copiés.
- Le **cache** garde des données près du processeur ; bien les réutiliser peut réduire l’attente.
- Moins de paramètres ou de MAC ne garantit pas un calcul plus rapide : mesurer la vraie cible.

L’IDÉE À RETENIR

Une opération rapide ne sert pas à grand-chose si le processeur attend ses données.

Le calcul n’est pas seul.

Il faut déplacer les nombres.



Lire une fois, réutiliser plusieurs fois ?

Une piste à tester, selon les données et le matériel.

2020 : Lin, Chen et leur équipe adaptent le réseau et son exécution.

Avec MCUNet, le modèle et le moteur de calcul sont conçus ensemble pour le microcontrôleur.

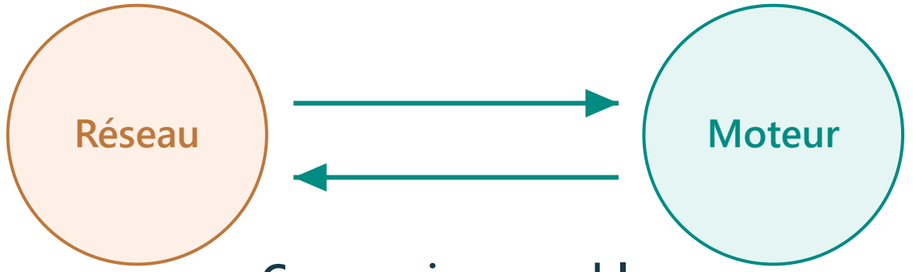
- Le **modèle** choisit les calculs à effectuer.
- Le **moteur** organise leur exécution et leur mémoire.
- MCUNetV2 traite des portions plus petites ; les recouvrements peuvent ajouter du calcul.

L'IDÉE À RETENIR

Le réseau, son programme et le matériel doivent être pensés ensemble.

2020 · MCUNet

Ji Lin, Wei-Ming Chen et leur équipe



Concevoir ensemble

Sur une carte donnée,
avec une mémoire et un délai donnés.

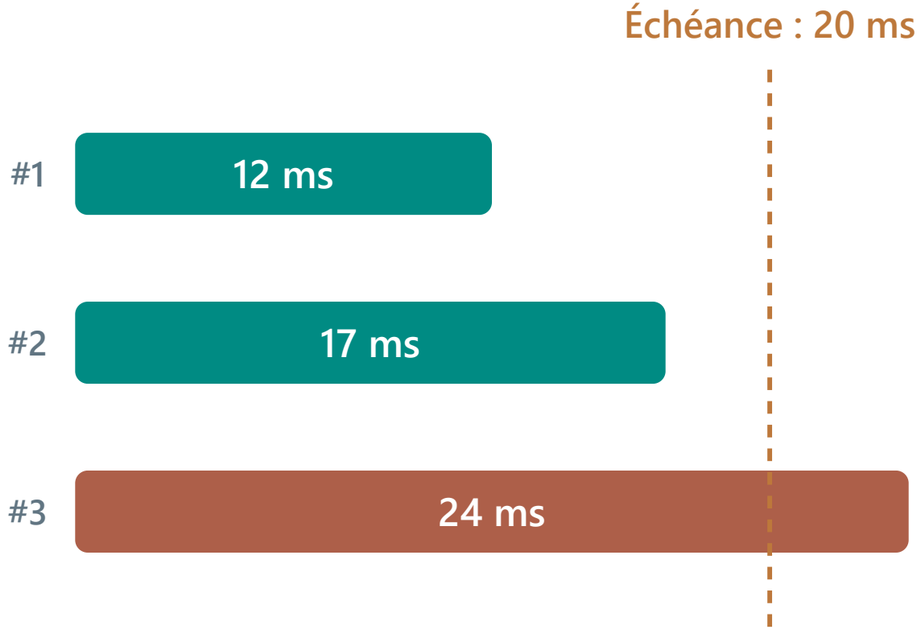
La lampe a compris. Mais elle répond trop tard.

Alice attend un service réactif, pas seulement une réponse correcte.

- La **latence** est le délai d'une réponse. Le **débit** est le nombre de réponses par seconde.
- La **gigue** décrit la variation des délais.
- L'**échéance** est le délai à respecter pour toute la chaîne : acquisition, préparation, calcul et action.

L'IDÉE À RETENIR

Une bonne réponse doit aussi arriver au moment où elle est utile.



Durées pédagogiques · le troisième appel est trop tardif.

Et sur batterie, combien de temps cela dure ?

Pour explorer ce problème, imaginons maintenant un appareil autonome sur batterie.

- La **puissance P**, en watts, est un rythme de consommation.
- L'**énergie E**, en joules, s'accumule sur une durée Δt : E = P × Δt si P reste constante.
- 0,1 watt pendant 0,02 seconde utilise 0,002 joule, soit 2 millijoules.

L'IDÉE À RETENIR

Il faut compter l'écoute, le calcul, les échanges et la veille, pas seulement le réseau de neurones.

La batterie paie toute la chaîne.

Écoute + calcul + échanges + veille



1. **Écouter** — Micro et acquisition
2. **Calculer** — Modèle et mémoire
3. **Échanger** — Radio et réseau

Deux fois plus rapide… pour quelle partie ?

La loi d'Amdahl aide à prévoir ce que gagne l'application entière.

- p est la fraction du temps initial passée dans la partie optimisée.
- s est l'accélération de cette partie. Gain global = 1 / [(1 − p) + p/s].
- Si p = 20 % et s = 2, le temps total baisse de 10 %. Le reste n'a pas changé.

L'IDÉE À RETENIR

Accélérer un petit morceau ne divise pas tout le temps par le même nombre.



Quel sera le nouveau délai total ?

L'application prenait 100 ms, dont 20 ms dans une brique. Seule cette brique devient deux fois plus rapide.

Avant : 100 ms au total

Autres étapes : 80 ms.

Brique à optimiser : 20 ms.



Le reste du programme conserve son délai. Ajoutez-lui celui de la brique optimisée.

POUR POURSUIVRE APRÈS LA RÉPONSE

- une conversion ajoute 12 ms. Le système reste-t-il plus rapide ?

A 90 ms

B 50 ms

C 80 ms

Essayons de mesurer, ici, pour de vrai.

Deux façons d’additionner les mêmes 4 096 valeurs dans votre navigateur.

- D’abord, vérifier que les deux programmes donnent le même résultat.
- Faire un échauffement, puis sept paires de mesures en alternant l’ordre.
- Exporter les observations. Elles décrivent ce navigateur, pas une carte embarquée.

L’IDÉE À RETENIR

Un chiffre utile vient avec ses conditions de mesure et ses données brutes.

NAVIGATEUR ACTUEL · SOMME DE 4 096 VALEURS

Pas encore mesuré. Aucune durée préremplie.

Par somme	Médiane	Min–max
Boucle	—	—
reduce	—	—

Résultat vérifié avant et après. Données : Float64Array, 32 Kio ; ceci n’est pas un pic RAM système.

Les mesures varient avec le navigateur et sa charge. Aucune énergie ni durée sur carte n’est déduite.

Chez Alice, ça marche. Et chez quelqu’un d’autre ?

La démonstration n’a pas encore essayé toutes les voix, tous les bruits ni toutes les pannes.

- Tester les entrées inhabituelles ; prévoir un refus ou une commande manuelle.
- Journaliser les erreurs et garder la version du programme, des poids et des réglages.
- Vérifier chaque mise à jour ; pouvoir revenir à une version qui fonctionne.

L’IDÉE À RETENIR

Un système fiable sait aussi quoi faire quand il ne sait pas répondre.



Essayer d’autres personnes, d’autres bruits, d’autres conditions.

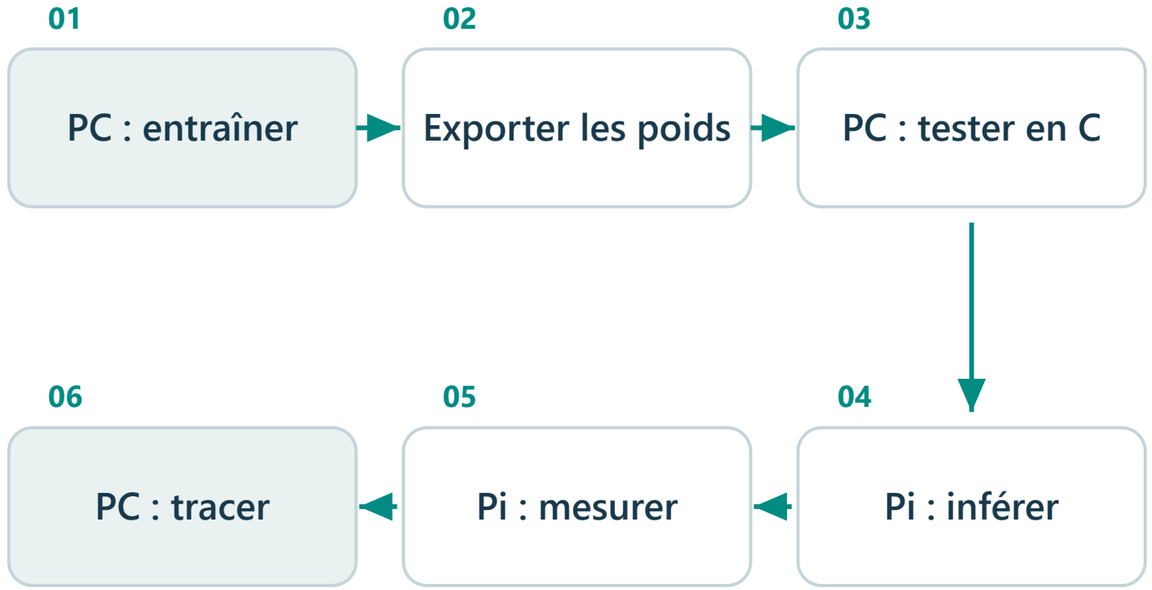
Du premier modèle à une expérience sur Pi.

Quatre TD : comprendre les fonctions, apprendre sur PC, puis déployer et améliorer.

- **Construire :** petits calculs sur papier, fonctions Python, tests puis apprentissage.
- **Déployer :** poids exportés, inférence C sur PC puis sur Pi, sorties comparées.
- **Enquêter :** une hypothèse, une variante sur PC, mesures sur Pi et courbes issues des CSV.

L'IDÉE À RETENIR

La question est précise : cette modification aide-t-elle sur notre cible, et pourquoi ?



Modifier sur PC → vérifier → redéployer → comparer

Aujourd'hui, que cherche-t-on encore à améliorer ?

Il reste à choisir le bon calcul, bien le représenter et l'adapter au matériel et à l'usage.

- **Architectures :** MobileNetV4 étudie des modèles pour plusieurs matériels mobiles.
- **Compression :** AWQ utilise les activations pour mieux quantifier les poids des modèles de langage.
- **Adaptation locale :** ne modifier que certains paramètres pour apprendre avec peu de mémoire.

L'IDÉE À RETENIR

Il n'existe pas de modèle optimal pour tous les appareils et tous les besoins.

Trois pistes parmi d'autres

Chaque résultat dépend de sa tâche et de sa cible.



Mieux calculer	Mieux représenter	Apprendre sur place
MobileNetV4	AWQ	Mises à jour partielles

Quel compromis pour votre appareil ?

Quel modèle respecte toutes les exigences ?

Exigences : qualité ≥ 96 %, RAM ≤ 100 Kio, p95 observé ≤ 20 ms et énergie ≤ 3 mJ par décision.

Valeurs fictives · même protocole

Modèle	Qualité	Kio	ms[1]	mJ
A	97 %	120	12	2,4
B	96 %	96	18	2,8
C	95 %	72	14	1,8

[1] p95 sur 500 essais fictifs. Énergie : acquisition + calcul + action, hors veille.

Un candidat doit satisfaire les quatre seuils, sans compensation entre eux.

POUR POURSUIVRE APRÈS LA RÉPONSE

- quel essai supplémentaire pourrait remettre cette décision en cause ?

A Le modèle A

B Le modèle B

C Le modèle C

Alice peut expliquer ce qu’il y a dans sa boîte.

Elle sait maintenant ce que l’appareil fait, ce qu’il coûte et ce qu’il reste à vérifier.

- Le **besoin** explique pourquoi cette fonction est utile.
- Les **mesures** disent si elle tient dans la mémoire, le délai et l’énergie disponibles.
- La **recherche** commence par une limite précise et une idée que l’on peut mettre à l’épreuve.

L’IDÉE À RETENIR

Embarquer l’IA, c’est transformer un modèle en un service qui fonctionne dans un appareil réel.



Reprenez les quatre questions du départ : vos réponses ont-elles changé ?

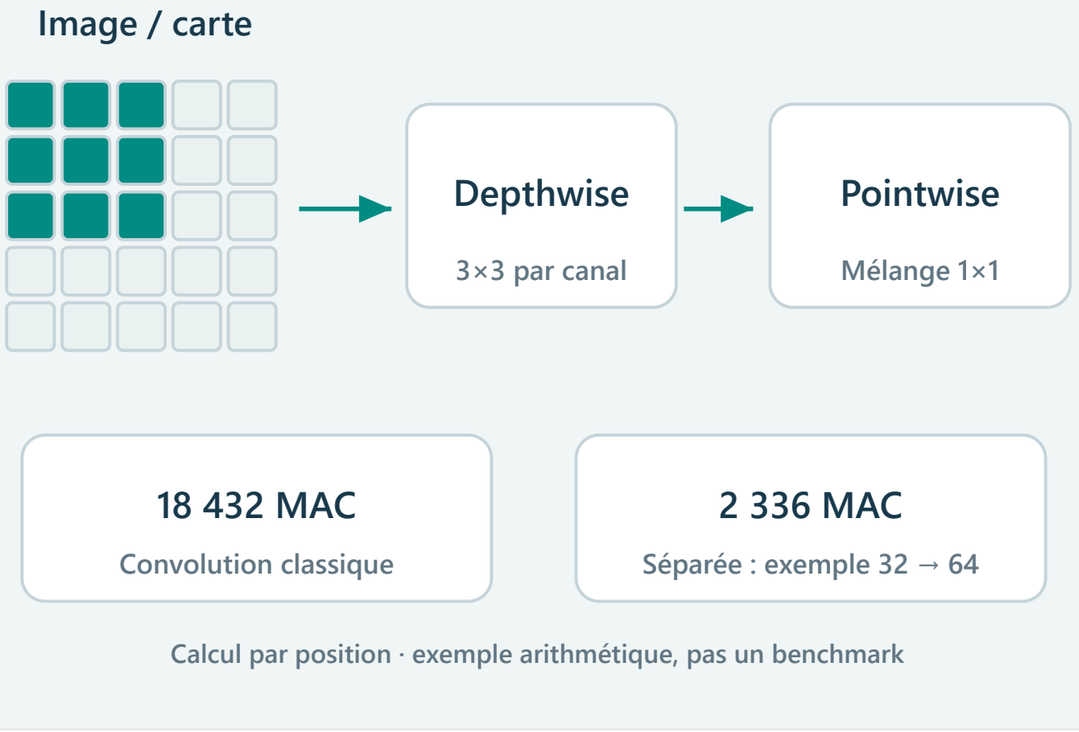
La convolution réutilise un petit motif.

Une convolution séparée découpe le filtrage spatial et le mélange des canaux.

> **Classique :** chaque sortie combine espace et canaux.

> **Depthwise + 1×1 :** filtrer chaque canal, puis les mélanger.

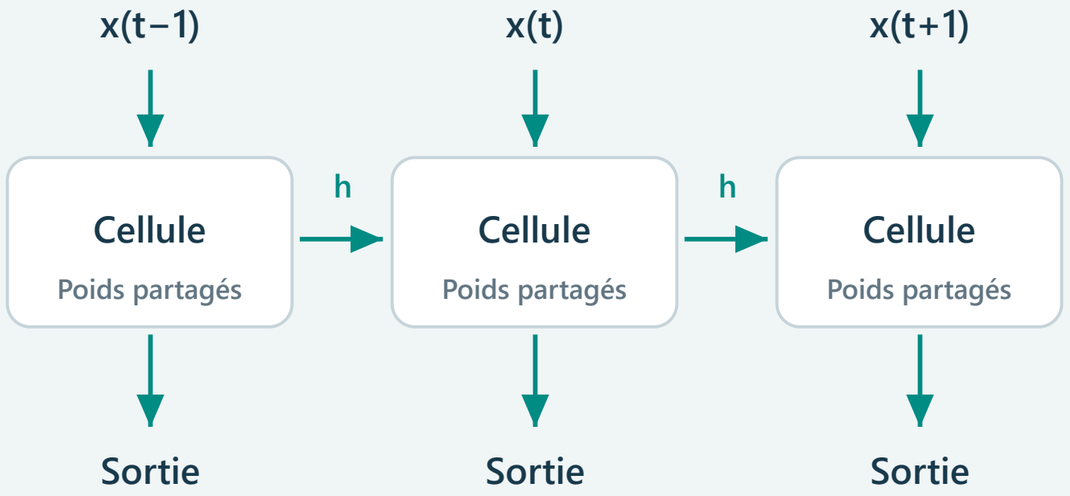
> **À vérifier :** moins de MAC peut encore produire plus de transferts.



Un état mémorise une partie du passé.

Un réseau récurrent met à jour un petit état à chaque échantillon.

- **État h :** résumé réutilisé au pas suivant.
- **Intérêt :** traiter un flux sans garder tout son historique.
- **Limite :** dépendance séquentielle, dérive et temps de mise en régime.



État transmis ≠ historique complet stocké

L’attention choisit quelles relations regarder.

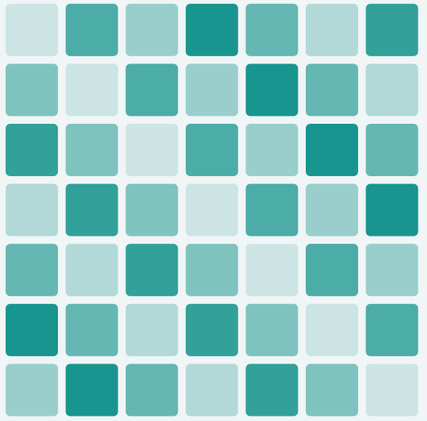
Au lieu d’un mélange fixe, les relations dépendent du contenu de l’entrée.

Q, K, V : requêtes, clés et valeurs calculées par projections.

Comparer : associer chaque position à d’autres positions.

Coût : une matrice naïve de scores grandit comme le carré de la longueur.

Scores entre positions



Normaliser puis mélanger

$\mathrm{softmax}(QK^{\intercal} / \sqrt{d})\, V$

$^{\intercal}$: transpose · d : dimension d’une tête

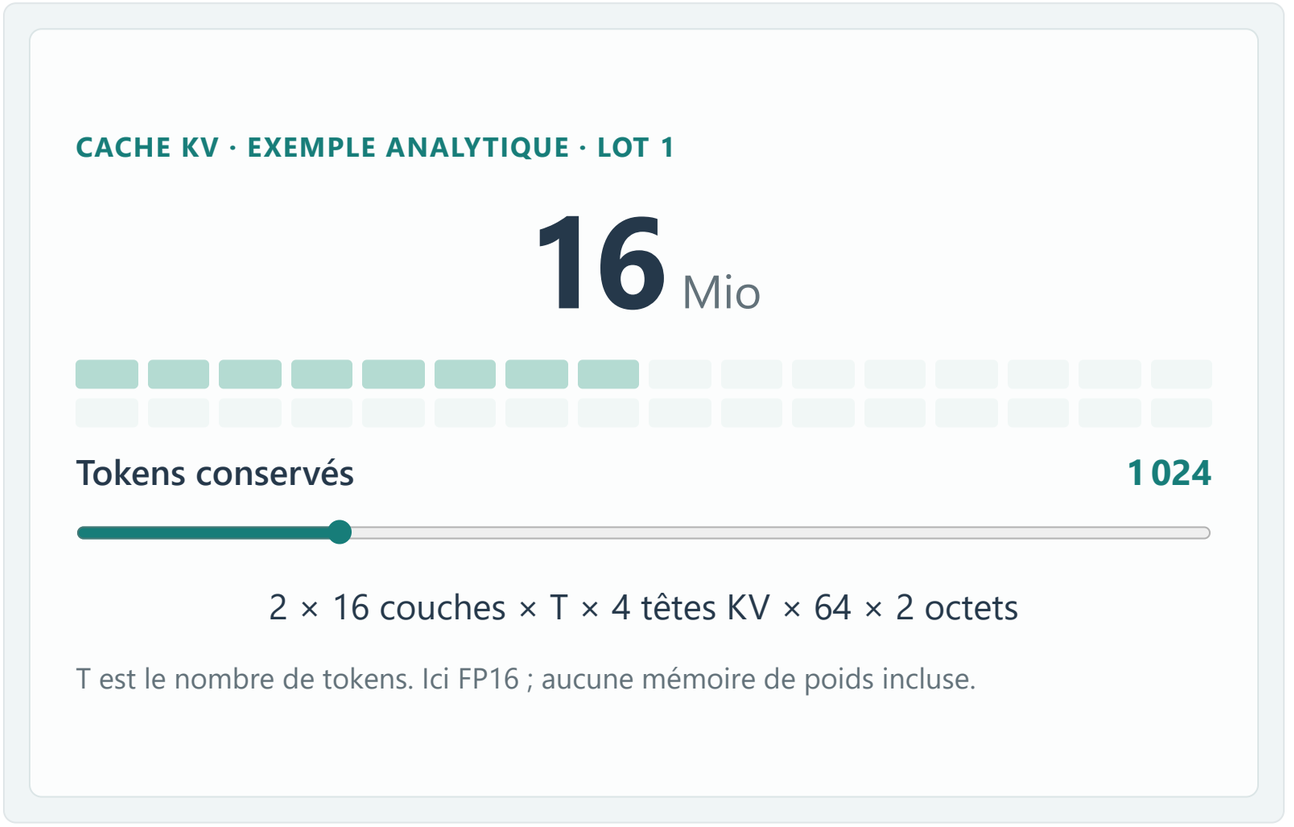
Doubler la longueur → 4× plus de scores naïfs

Carte d’attention illustrative ; aucune sortie de modèle mesurée

Le contexte consomme aussi de la mémoire.

Un cache KV évite de recalculer d'anciennes clés et valeurs lors de la génération.

- **Préremplissage :** traiter la consigne initiale.
- **Décodage :** produire ensuite un token à la fois.
- **Conséquence :** réduire les poids ne réduit pas automatiquement le cache.



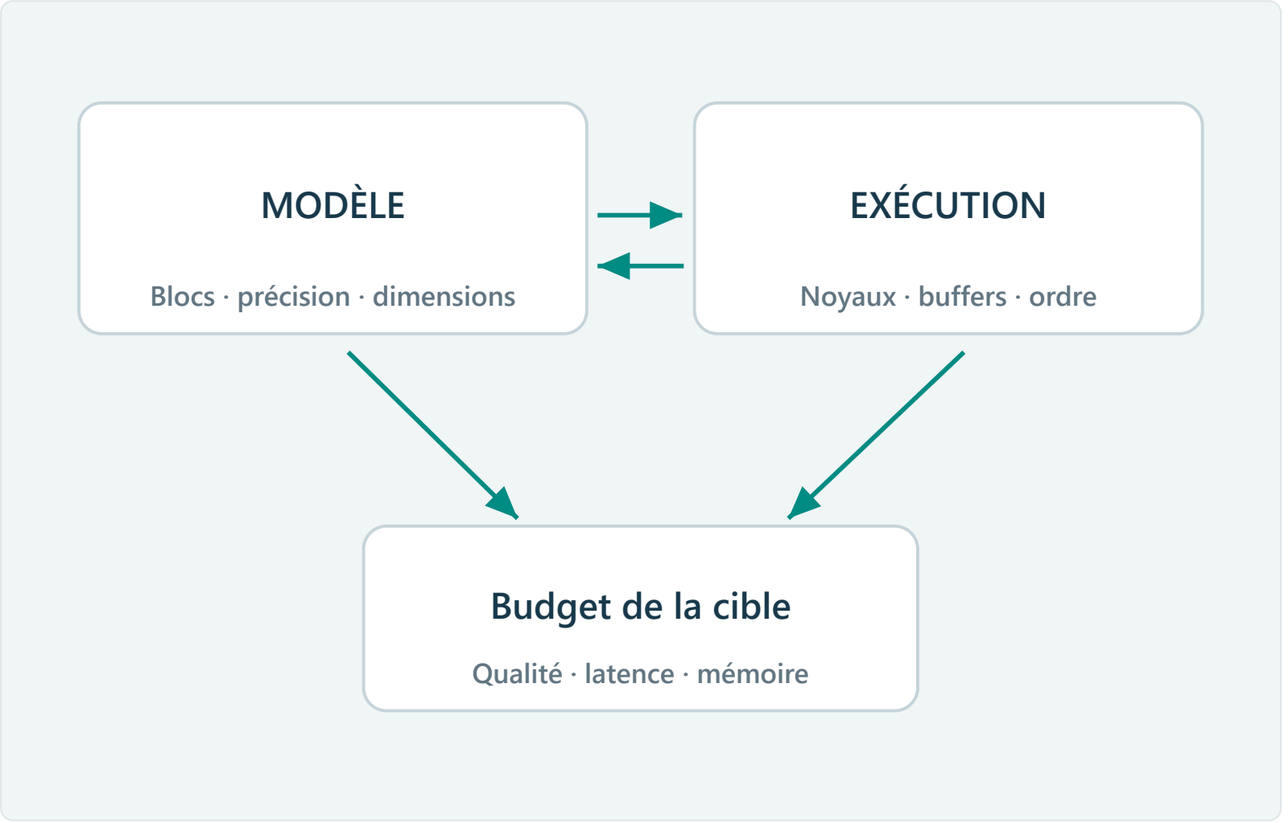
Deux recherches : déplacer la contrainte au bon endroit.

Architecture et exécution se répondent ; une seule métrique masque le compromis.

> **MCUNetV2 · 2021 :** calculer par morceaux pour réduire le pic mémoire.

> **MobileNetV4 · 2024 :** concevoir des blocs efficaces pour plusieurs cibles mobiles.

> **Question ouverte :** le gain reste-t-il vrai sur notre cible et notre tâche ?



Calibrer, entraîner, puis comparer le vrai calcul.

La meilleure précision n'est pas forcément uniforme dans tout le réseau.

PTQ : quantifier après entraînement, avec calibration si nécessaire.

QAT : simuler les effets de quantification pendant l'apprentissage.

AWQ · 2024 : protéger les canaux sensibles à partir des activations.



Préserver ce qui change le plus la sortie

Puis vérifier qualité et noyau réellement exécuté.

Supprimer, simplifier, ou apprendre à imiter.

Compresser un fichier ne suffit pas à accélérer son exécution.

> **Élagage :** retirer poids ou canaux ; exploiter réellement la structure obtenue.

> **Distillation :** entraîner un petit élève à reproduire un enseignant.

> **Choix d'architecture :** chercher un compromis compatible avec la cible.



Le noyau doit exploiter la structure restante.

Distillation : enseignant → petit élève à entraîner

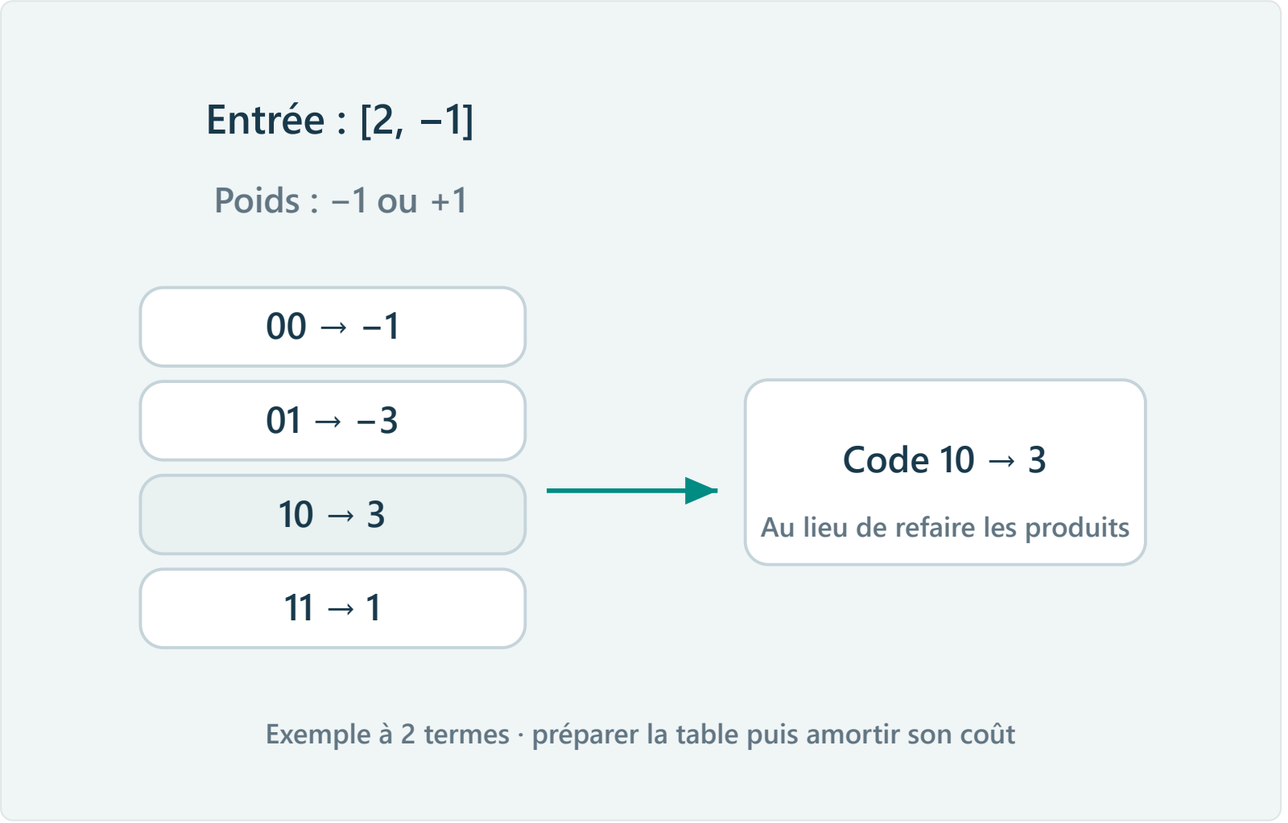
T-MAC · Remplacer un calcul par une consultation de table.

Jianyu Wei et al. · EuroSys 2025 · Une idée système pour produits bas bits sur CPU.

> **Problème :** les poids bas bits peuvent coûter cher à déquantifier.

> **Idée :** préparer des combinaisons, puis les retrouver par indices.

> **Limite :** la table et ses accès doivent être moins coûteux que le calcul évité.



« Petit modèle de langage » ne signifie pas « modèle pour MCU ».

Le mot petit dépend du point de comparaison.

> **1,7 milliard de paramètres à 4 bits :** environ 850 Mo de poids bruts.

> **À ajouter :** métadonnées, activations, cache KV et moteur.

> **À décider :** besoin réel de génération, ou classification spécialisée ?



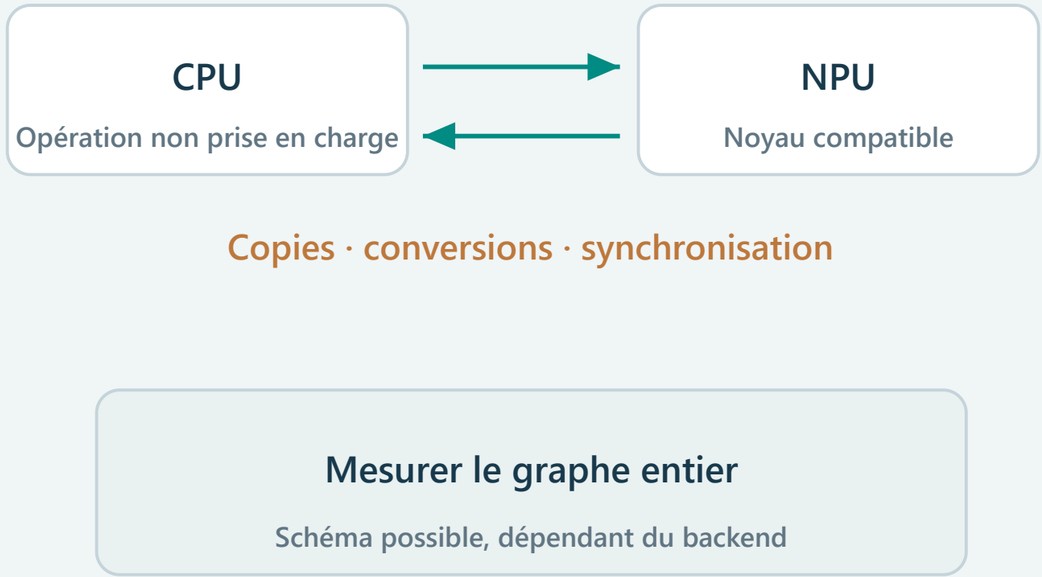
Veille 2025 · Le NPU ne gagne que si le graphe lui convient.

T-MAN · Wei, Li et al. · Prépublication : lire ce qui est accéléré, et ce qui ne l'est pas.

> **Problème :** des opérations périphériques limitent un accélérateur spécialisé.

> **Piste :** adapter tables, tuilage et pipeline.

> **À contrôler :** préremplissage, décodage, transferts et comparaison de départ.



Veille 2026 · Deux voies, deux niveaux d'intervention.

Des pistes actuelles à examiner ; aucune promesse universelle de performance.

> **FastGRNN · Kizilates :** très petite récurrence, tables et validation sur MCU.

> **RISC-V · Kumar et al. :** instructions et compilation pour Transformers compacts.

> **Regard critique :** mise en régime, graines, mémoire, aire et protocole.

JUIN 2026

FastGRNN sur MCU

Mise en régime
à examiner

Emre Can Kizilates

OCTOBRE 2026

Transformer sur RISC-V

Instructions + compilation
Aire et énergie à compter

Ajay Kumar M et al.

Deux prépublications : hypothèses et périmètres à vérifier.

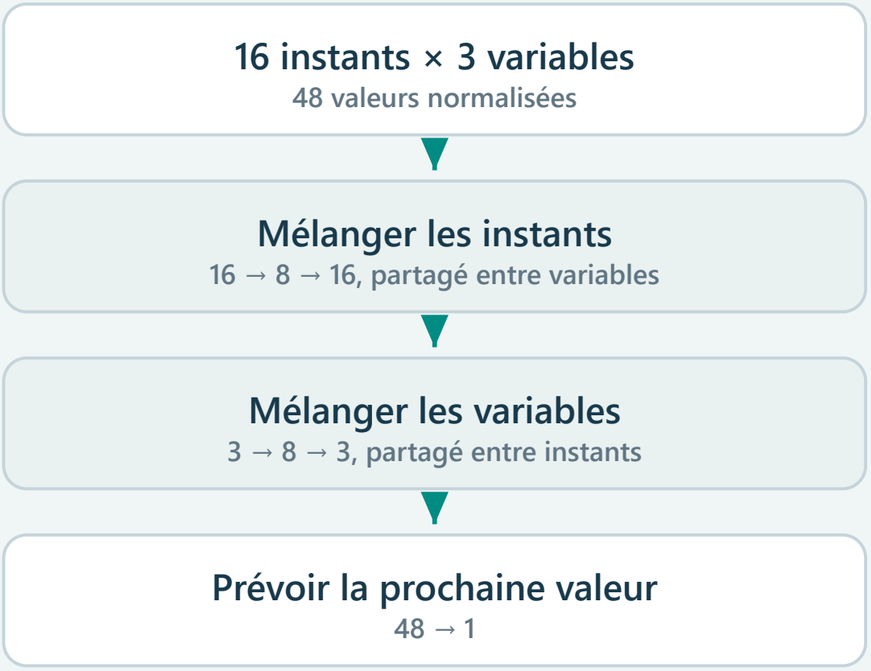
Du petit exemple au réseau du TD.

Le TD utilise un bloc de mélange temporel et de variables, inspiré de TSMixer. Il ne reconnaît pas la voix.

16 instants × 3 variables : chaque exemple contient 48 nombres normalisés. La cible est la prochaine valeur de la première variable.

Mélanger : 16 → 8 → 16 pour chaque variable ; puis 3 → 8 → 3 pour chaque instant. Les calculs sont partagés et ajoutent une correction à leur entrée.

Prédire : les 48 résultats deviennent une sortie. Le sujet définit les 388 paramètres ; cela représente 1 552 octets de poids en float32.



Chaque mélange ajoute sa correction à son entrée.

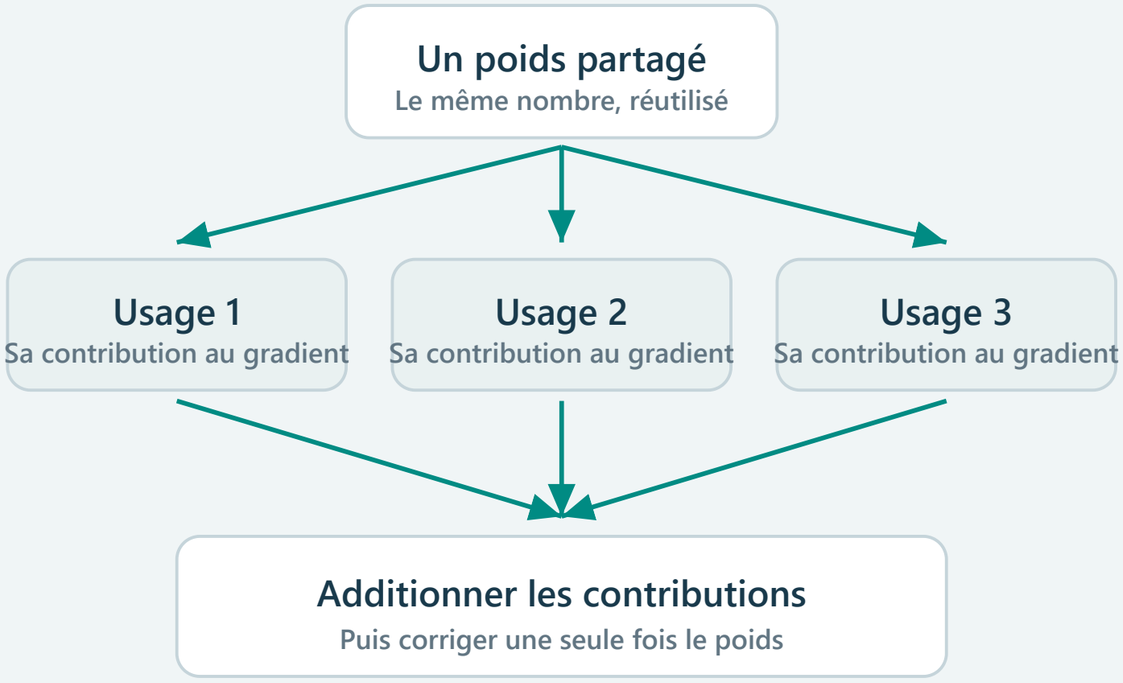
Apprendre avec plusieurs usages d’un même poids.

La correction d’un poids de sortie expliquée dans l’histoire est un premier pas, pas toute la rétropropagation du TD.

Remonter l’erreur : la règle de chaîne relie l’effet de chaque fonction à la perte. Une ReLU bloque le gradient si son entrée est négative.

Deux routes : dans une addition résiduelle, le gradient est transmis aux deux branches. Pour un poids réutilisé, additionner toutes ses contributions.

Vérifier avant d’entraîner : comparer les dérivées aux différences finies en double précision ; puis faire baisser la perte sur quelques exemples.



Sources et limites de lecture · 1/6

Articles originaux, prépublications et documentation officielle · lire les limites de chaque source.

R01 ALVINN: An Autonomous Land Vehicle In a Neural Network ↗
Dean Pomerleau · 1989 · Repère historique · publication CMU / actes NeurIPS
Relier perception apprise et action à bord d'un véhicule.

R02 MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications ↗
Andrew G. Howard et al. · 2017 · Article fondateur · arXiv 1704.04861
Séparer convolution spatiale et mélange des canaux pour réduire le calcul.

R03 Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference ↗
Benoit Jacob et al. · 2018 · CVPR 2018 · prépublication 2017
Co-concevoir quantification, apprentissage et noyaux entiers.

R04 MCUNet: Tiny Deep Learning on IoT Devices ↗
Ji Lin, Wei-Ming Chen et al. · 2020 · NeurIPS 2020
Chercher ensemble l'architecture du réseau et son moteur d'exécution.

Sources et limites de lecture · 2/6

Articles originaux, prépublications et documentation officielle · lire les limites de chaque source.

R05 MCUNetV2: Memory-Efficient Patch-based Inference for Tiny Deep Learning ↗
Ji Lin, Wei-Ming Chen, Han Cai et al. · 2021 · NeurIPS 2021
Traiter des morceaux de carte de caractéristiques pour réduire le pic mémoire.

R06 On-Device Training Under 256KB Memory ↗
Ji Lin, Ligeng Zhu et al. · 2022 · NeurIPS 2022
Rendre possible une adaptation locale par mises à jour sélectives et co-conception.

R07 FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness ↗
Tri Dao, Daniel Y. Fu et al. · 2022 · NeurIPS 2022
Réorganiser le calcul d'attention pour réduire les échanges mémoire sur GPU.

R08 TSMixer: An All-MLP Architecture for Time Series Forecasting ↗
Si-An Chen et al. · 2023 · Transactions on Machine Learning Research, 2023
Mélanger les instants puis les variables avec des perceptrons multicouches.

Sources et limites de lecture · 3/6

Articles originaux, prépublications et documentation officielle · lire les limites de chaque source.

R09 AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration ↗
Ji Lin, Jiaming Tang et al. · 2024 · MLSys 2024 · prépublication 2023
Utiliser des statistiques d'activation pour mieux préserver les canaux sensibles.

R10 MobileNetV4 — Universal Models for the Mobile Ecosystem ↗
Danfeng Qin et al. · 2024 · ECCV 2024
Concevoir des blocs et une recherche d'architecture adaptés à plusieurs matériels mobiles.

R11 T-MAC: CPU Renaissance via Table Lookup for Low-Bit LLM Deployment on Edge ↗
Jianyu Wei, Shijie Cao et al. · 2025 · EuroSys 2025 · prépublication 2024
Remplacer certaines opérations de produits mixtes par des tables de recherche adaptées.

R12 SmolLM2: When Smol Goes Big — Data-Centric Training of a Small Language Model ↗
Loubna Ben Allal et al. · 2025 · Prépublication arXiv 2502.02737
La qualité et le mélange des données comptent aussi pour les petits modèles de langage.

Sources et limites de lecture · 4/6

Articles originaux, prépublications et documentation officielle · lire les limites de chaque source.

R13 T-MAN: Enabling End-to-End Low-Bit LLM Inference on NPUs via Unified Table Lookup ↗
Jianyu Wei, Qingtao Li et al. · 2025 · Prépublication arXiv · 14 novembre 2025
Adapter tables, tuilage et pipeline aux opérations réellement prises en charge par un NPU.

R14 From Compression to Deployment: Real-Time and Energy-Efficient FastGRNN on Ultra-Constrained Microcontrollers ↗
Emre Can Kizilates · 2026 · Prépublication arXiv · 15 juin 2026
Étudier ensemble compression, activation par table, état récurrent et déploiement physique.

R15 A Framework for Accelerating Transformer Inference on RISC-V for Edge AI ↗
Ajay Kumar M et al. · 2026 · Prépublication arXiv · accepté à ICECS 2026 selon la notice des auteurs
Adapter instructions et compilation à des Transformers compacts, principalement encodeurs.

R16 Adversarial Machine Learning: A Taxonomy and Terminology of Attacks and Mitigations ↗
Apostol Vassilev, Alina Oprea et al. · 2025 · Rapport technique NIST AI 100-2e2025
Distinguer attaques sur données, entrées, modèles et vie privée.

Sources et limites de lecture · 5/6

Articles originaux, prépublications et documentation officielle · lire les limites de chaque source.

R17 Tiny Machine Learning: Progress and Futures ↗
Ji Lin, Ligeng Zhu, Wei-Ming Chen, Wei-Chen Wang, Song Han · 2023 / 2024 · IEEE Circuits and Systems Magazine 2023 · dépôt arXiv 2024
Vue d'ensemble de la co-conception et du passage de l'inférence à l'adaptation locale.

R18 MLPerf Tiny Benchmark ↗
Colby Banbury, Vijay Janapa Reddi et al. · 2021 · NeurIPS 2021 · Datasets and Benchmarks
Comparer qualité, latence et énergie avec un protocole reproductible.

R19 CMSIS-NN — Efficient neural network kernels for Arm Cortex-M ↗
Arm · Documentation consultée en 2026 · Dépôt officiel de noyaux optimisés
Les performances dépendent de noyaux adaptés à l'ISA et aux formats quantifiés.

R20 LiteRT for Microcontrollers ↗
Google AI Edge · Documentation consultée en 2026 · Documentation officielle du moteur MCU
Déployer un sous-ensemble d'opérateurs avec une allocation adaptée au microcontrôleur.

Sources et limites de lecture · 6/6

Articles originaux, prépublications et documentation officielle · lire les limites de chaque source.

R21 MLPerf Inference: Tiny ↗
MLCommons · Documentation consultée en 2026 · Méthodologie et résultats officiels
Lire conjointement qualité, scénario de mesure et configuration matérielle.

R22 Shakey the Robot ↗
SRI International · 1966–1972 · Histoire du projet · organisme concepteur
Faire percevoir un environnement et planifier des actions à un robot mobile.

R23 Edge AI ↗
NIST · Documentation consultée en 2026 · Programme de recherche officiel
Rapprocher l'inférence ou l'apprentissage des données et des usages.

R24 Glossaire structuré de l'IA embarquée ↗
Embedded France avec la Fondation Eclipse · 2026 · Référentiel pédagogique · v1.1, 22 juillet 2026 · CC BY 4.0
Relier données, modèles, exécution, matériel, mesures et cycle de vie.

Lexique · Matériel et mémoire

Des termes à expliquer avec un exemple, pas seulement à mémoriser.

MCU

Microcontrôleur : processeur, mémoire et périphériques réunis pour une application embarquée.

CPU / GPU / NPU

Calcul général / calcul parallèle / accélération spécialisée de réseaux.

Flash

Mémoire non volatile : programme et constantes, selon le placement choisi.

SRAM / RAM

Mémoire de travail volatile. Compter les données simultanément vivantes.

SIMD

Une instruction traite plusieurs valeurs ; les formats et alignements comptent.

DMA

Transfert de données piloté par un contrôleur ; bande passante et synchronisation restent limitées.

ISA

Jeu d'instructions du processeur. Un noyau doit exploiter ce qui est réellement disponible.

Ko / Kio

Ko = 1000 octets ; Kio = 1024 octets. Mio = 1024^2 octets dans ce cours.

Lexique · Modèles et expérimentation

Des termes à expliquer avec un exemple, pas seulement à mémoriser.

Paramètre / activation

Un paramètre est appris ; une activation dépend de l'entrée en cours de calcul.

MAC

Multiplier puis accumuler. Préciser la convention avant de comparer des opérations.

Perte / gradient

La perte mesure une erreur ; le gradient indique comment elle varie avec les paramètres.

PTQ / QAT

Quantification après entraînement / apprentissage tenant compte de la quantification.

Cache KV

Clés et valeurs conservées pour éviter certains recalculs pendant la génération.

Runtime / backend

Moteur qui exécute le graphe / implémentation qui choisit les noyaux de la cible.

Latence / débit

Temps d'une réponse / quantité de réponses par unité de temps. Ils ne se confondent pas.

CI / manifeste

Tests et construction automatisés / fiche qui relie version, conditions et fichiers de l'expérience.

Réponses raisonnées · 1/2

L'argument compte davantage que la lettre choisie.

Questions 1 · réponse B · diapositive 7

Des voix gardées à part permettent de tester de nouvelles personnes. Réussir sur les exemples déjà appris ne suffit pas.

Questions 2 · réponse C · diapositive 12

$h_1 = 1$ et $h_2 = 1{,}5$. Donc $\hat{y} = 2 \times 1 + 1{,}5 = 3{,}5$. Les poids n'ont pas changé.

Questions 3 · réponse A · diapositive 17

A respecte les deux exigences. Avec B, l'audio quitte l'objet. Avec C, la commande dépend aussi du réseau.

Réponses raisonnées · 2/2

L'argument compte davantage que la lettre choisie.

Questions 4 · réponse B · diapositive 24

60 + 40 + 20 + 16 = 136 Kio. Les poids restent ici en Flash. Avec 128 Kio de RAM, il manque au moins 8 Kio.

Questions 5 · réponse A · diapositive 30

Le reste prend toujours 80 ms. La brique passe de 20 à 10 ms : total 90 ms, soit 10 % de temps économisé.

Questions 6 · réponse B · diapositive 35

B respecte les quatre seuils. A dépasse la RAM ; C n'atteint pas la qualité demandée. Ce choix reste à valider dans d'autres conditions.